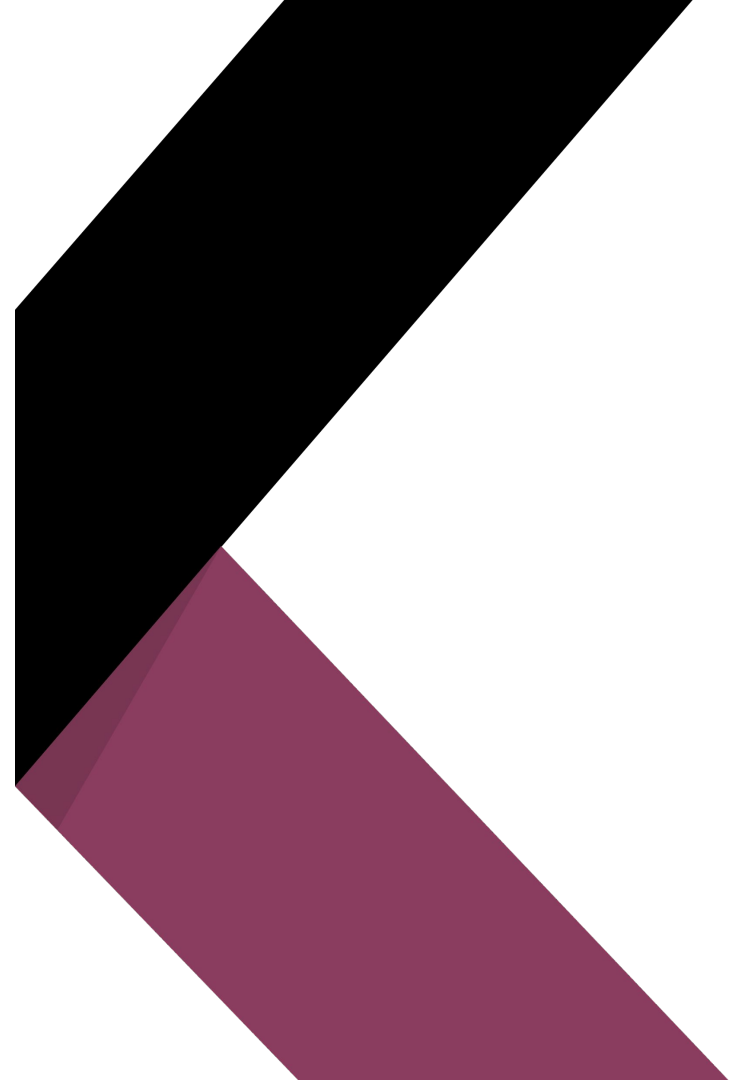


Network - Analysys

CAN-LIN-Ethernet





CAN Network



CAN - Introduction

The CAN (Controller Area Network) bus is broadcast digital bus designed to operate at speeds from 20kb/s to 1Mb/s. The transmission rate depends on the bus length and transceiver speed.

The CAN network protocol has been defined to provide deterministic communication in complex distributed systems with the following features/capabilities:

- Possibility of assigning priority to messages and guaranteed maximum latency times.
- Multicast communication with bit-oriented synchronization.
- System wide data consistency.
- Multimaster access to the bus.
- Error detection and signalling with automatic retransmission of corrupted messages.
- Detection of possible permanent failures of nodes and automatic switching off of defective nodes.

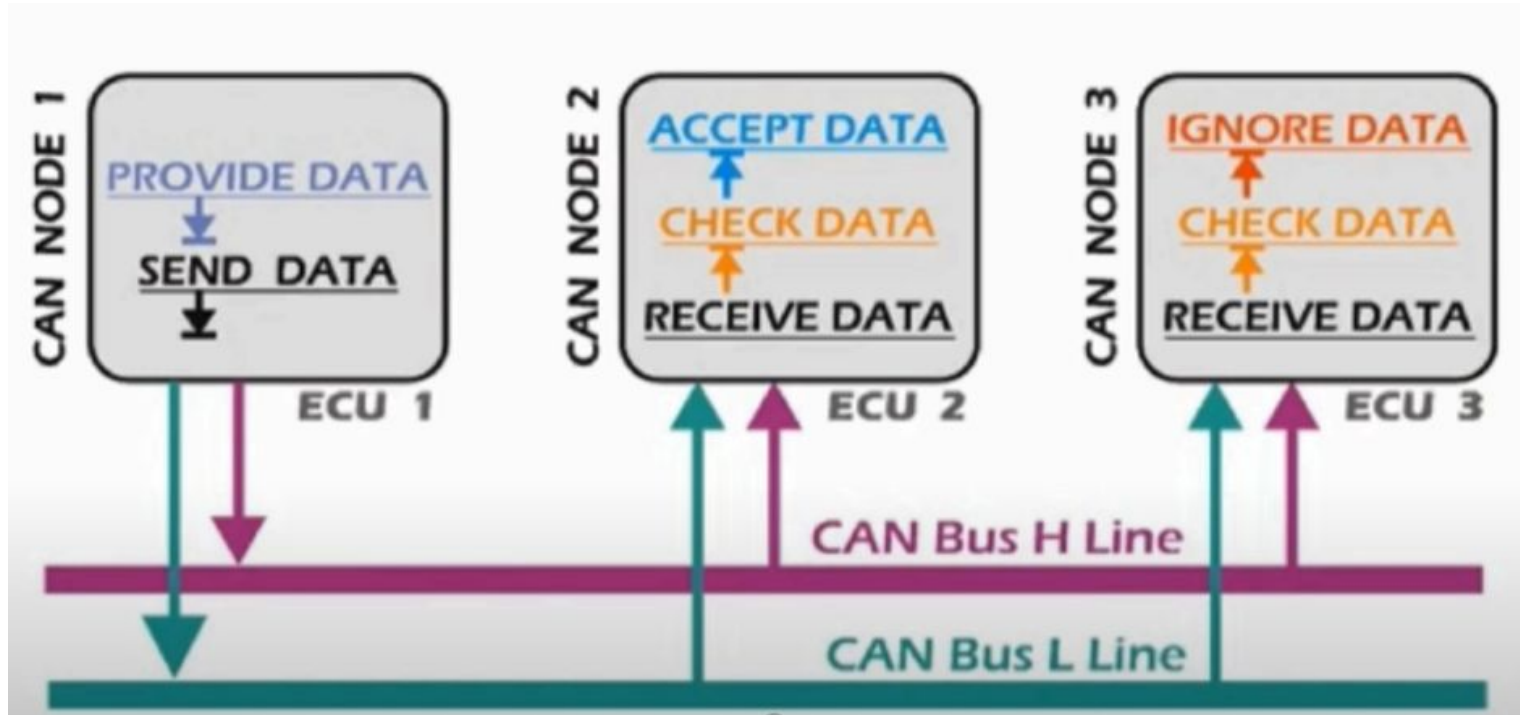


CAN - Network

- The interface between a CAN controller chip and a two-wire differential bus typically consists of a transmitting and a receiving amplifier (transceiver = transmit and receive).
- The transceiver must convert the electrical representation of a bit from the one in use by the controller to the one defined for the bus.
- In addition, it must provide sufficient output current to drive the bus electrical state, and protect the controller chip against overloading.
- As a receiver, the CAN transceiver provides the recessive signal level and protects the controller chip input comparator against excessive voltages on the bus lines.
- Furthermore, it detects bus errors such as line breakage, short circuits, shorts to ground, etc.
- A further function of the transceiver can also be the galvanic isolation between a CAN node and the bus line.



CAN - CAN DATA TRANSFER PROCESS





CAN - Transmission rate & Bus length

- Because of the definition of the bit time, it clearly exists a dependency between the bit time and the signal propagation delay, that is, between the maximum achievable bit rate (or transmission speed) and the length of the bus
- The signal propagation delay to be considered for the computation of the maximum allowed bus length includes several stages
- For short, high speed networks, the biggest limitation to bus length is the transceivers propagation delay.

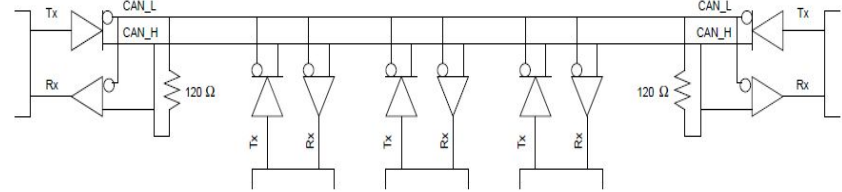
Bit rate	bit time	bus length
1 Mb/s	1 μ s	25m
800 kb/s	1.25 μ s	50m
500 kb/s	2 μ s	100m
250 kb/s	4 μ s	250m
125 kb/s	8 μ s	500m
62.5 kb/s	16 μ s	1000m
20 kb/s	50 μ s	2500m
10 kb/s	100 μ s	5000m



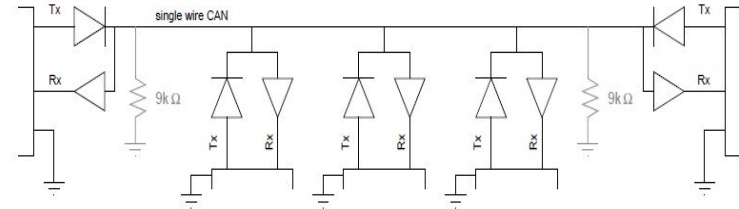
CAN - Bus termination

Electrical signals on the bus are reflected at the ends of the electrical line unless measures are taken. For the node to read the bus level correctly it is important that signal reflections are avoided. This is done by terminating the bus line with a termination resistor at both ends of the bus and by avoiding unnecessarily long stubs lines of the bus

- For high speed CAN, ISO 11898 requires a cable with a nominal impedance of 120 ohms, and therefore 120 ohm resistors should be used for termination (only the devices on the ends of the cable need termination resistors).
- For Low-Speed CAN, each device on the network needs a termination resistor for each data line.



Terminating a High-Speed Network.

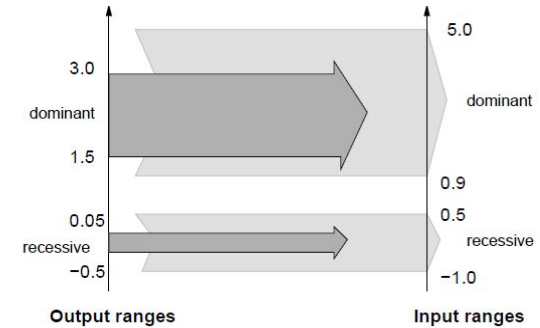
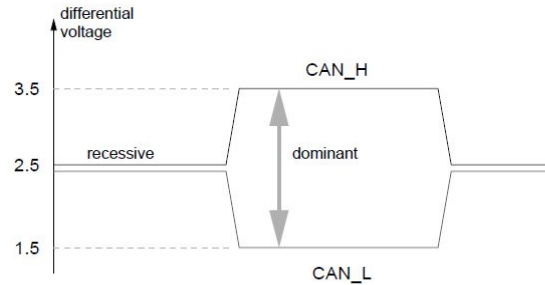


Termination Resistors on a Low-Speed Network.

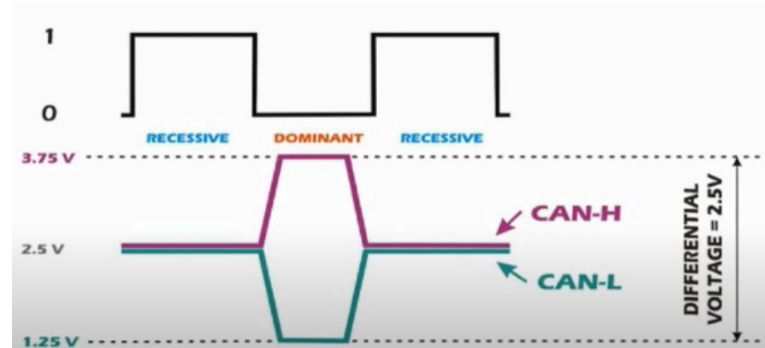


CAN - Physical encoding of Dominant and recessive states

- CAN specifies two logical states: recessive and dominant. According to ISO-11898-2, a differential voltage is used to represent recessive and dominant states.
- In the recessive state (usually logic 1), the differential voltage on CAN H and CAN L is less than the minimum threshold ($<0.5V$ receiver input or $<1.5V$ transmitter output). In the dominant state (logic 0), the differential voltage on CAN H and CAN L is greater than the minimum threshold.



Encoding of dominant and recessive bits.





CAN - Message Format

In CAN, there are 4 different types of frames, according to their content and function:

- **DATA FRAMES** contain data information from a source to possibly multiple receivers.
- **REMOTE FRAMES** are used to request transmission of a corresponding (with the same identifier) DATA FRAME.
- **ERROR FRAMES** are transmitted whenever a node on the network detects an error.
- **OVERLOAD FRAMES** are used for flow control, to request an additional time delay before the transmission of a DATA or REMOTE frame.

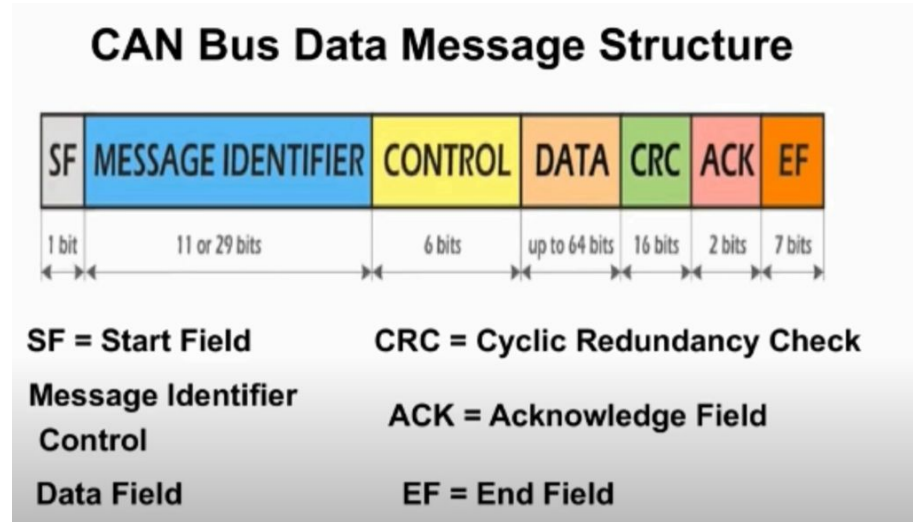


CAN - Data frame

DATA frames are used to transmit information between a source node and one of more receivers. CAN frames do not use explicit addressing for identifying the message receivers, but each node defines the messages that will be received based on their information content, which is encoded in the IDENTIFIER field of the frame.

The structure of the DATA frames is:

- Identifier field;
- Control field;
- Data field;
- CRC and acknowledgment field;
- Interframe space.



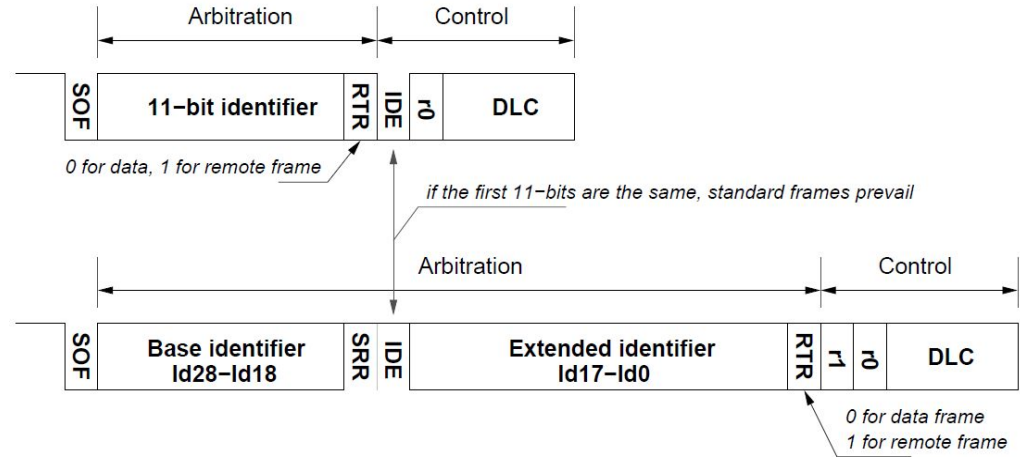


CAN - Data frame (Identifier)

The CAN protocol requires that all contending messages have a unique identifier. The identifier field consists of 11 (+1) bits in **standard** format and 29 (+3) bits in **extended** format.

The field is built:

- the first 11 bits of the identifier;
- RTR (Remote Transmission Request) bit in the standard and extended format;
- SRR (Substitute Remote Request), only in extended format; it's only a place holder
- Single IDE bit (Identifier Extension, always recessive)
- the remaining 18 least significant identifier bits





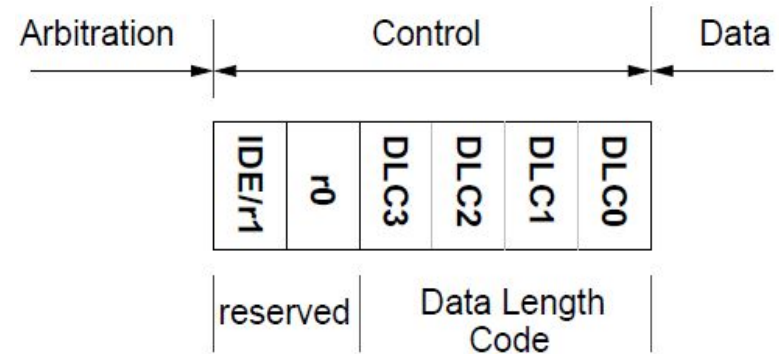
CAN - Data frame (Control, Data and Interframe space)

The **control field** contains 6 bits. The first two bits are reserved or predefined in content.

The following four bits define the length of the data content (Data Length Content, or DLC) in bytes. If the dominant bit is interpreted as 1 (contrary to the common notation in which it is read as 0) and the recessive as 0, the four DLC bits are the unsigned binary coding of the length.

The **data field** position is on the right side of the control field. The maximum dimension of the data field is 8 bytes.

Data frames and remote frames are separated by an **INTERFRAME** space (7 recessive bits) on the bus.





CAN - Remote/Error/Overload frame

A **remote** frame is used to request the transmission of a message with a given identifier from a remote node. A remote frame has the same format of a data frame with the following characteristics:

- the identifier field is used to indicate the identifier of the requested message
- the data field is always empty (0 bytes)
- the DLC field indicates the data length of the requested message (not the transmitted one)
- the RTR bit in the arbitration field is always set to recessive

The **error** frame is not a true frame, but rather the result of an error signalling and recovery. The details of such an event are described in the following sections.

The ERROR FRAME and the **OVERLOAD** FRAME are of fixed form as well and not coded by the method of bit stuffing



CAN - Error management

The CAN protocol is designed for safe data transfers and provides mechanisms for **error detection**, signalling and self-diagnostics, including measures for fault confinement, which prevent faulty nodes from affecting the status of the network.

The general measures for error detection are based on the capability of each node of monitoring broadcast transmissions over the bus, whether it is a transmitter or a receiver, and to signal error conditions resulting from several sources. Corrupted messages are flagged by any node detecting an error. Such messages are aborted and will be retransmitted automatically.

The protocol requires that receivers **acknowledge** reception of the message by changing the content of the ACK FIELD. The ACK FIELD is two bits long and contains the ACK SLOT and the ACK DELIMITER. In the ACK FIELD the transmitting station sends two recessive bits. All receiver nodes that detect a correct message (after CRC checks), inform the sender by changing the recessive bit of the ACK SLOT into a dominant bit.



CAN - CRC checks

The CRC portion of the frame is obtained by selecting the input polynomial (to be divided) as the stream of bits from the START OF FRAME bit (included) to the DATA FIELD (if present) followed by 15 zeros.

This polynomial is divided by the generator:

$$X^{15} + X^{14} + X^{10} + X^8 + X^7 + X^4 + X^3 + 1.$$

The remainder of the division is the CRC SEQUENCE portion of the frame



CAN - Error types

There are 5 different error types:

- **BIT ERROR:** a unit that is sending a bit on the bus also monitors the bus. A BIT ERROR has to be detected at that bit time, when the bit value that is monitored is different from the bit value that is sent. Exceptions are the recessive bits sent as part of the arbitration process or the ACK SLOT.
- **STUFF ERROR:** a STUFF ERROR is detected at the 6th consecutive occurrence of the same bit in a message field that is subject to stuffing.
- **CRC ERROR:** if the CRC computed by the receiver differs from the one stored in the message frame.
- **FORM ERROR:** when a fixed-form bit field contains one or more illegal bits. (Note, that for a Receiver a dominant bit during the last bit of END OF FRAME is not treated as FORM ERROR, this is the possible cause of inconsistent message omission and duplicates).
- **ACKNOWLEDGMENT ERROR:** detected by a transmitter if a recessive bit is found on the ACK SLOT.



CAN - Error Confinement Mechanisms

Each node on network has two error counters: the **Transmit Error Counter** and the **Receive Error Counter**.

There are several rules governing how these counters are incremented and/or decremented.

A transmitter detecting a fault increments its Transmit Error Counter +8 for each corrupted message, and decrements it of 1 for each correctly send message.

- A node starts out in Error Active mode. When any one of the two Error Counters raises above 127, the node will enter a state known as **Error Passive** and when the Transmit Error Counter raises above 255, the node will enter the **Bus Off state**.
- An Error Active node will transmit Active Error Flags when it detects errors.
- An Error Passive node will transmit Passive Error Flags when it detects errors.
- A node which is Bus Off will not transmit anything on the bus at all.



CAN - Network Specification DBC

The Network specification, that is used to decode raw messages on CAN, into physical values and signals, is the **DBC File – Data Base Can**

Each CAN network has a dedicated DBC file.

In the DBC file are listed all the ECUs present on Network, and all the received and transmitted messages.

For each message is specified the transmission method (cyclic, on event..), the DLC, the identification of the message, the cycle time and the signals included in the message.

The screenshot shows a software interface for editing a DBC file. It features a tabbed menu at the top with options: Definition, Signals, Transmitters, Receivers, Layout, Attributes, and Comment. The 'Definition' tab is currently selected. Below the tabs, the following fields are visible:

- Name:** AIRBAG2
- Type:** CAN Standard
- ID:** 0x5A7
- DLC:** 1
- Transmitter:** NAB
- Tx Method:** Cyclic
- Cycle Time:** 1000



CAN - Network Specification DBC

List of the ECUs present on network

Vector CANdb++ Editor - \Gt.ferlan\Itadit\SEE\SW_Diagnosi\DBC\F173\DB_SR_CICAN_F173_2021_01_18_Rev276.dbc - [Overall View]

File Edit View Options Window Help

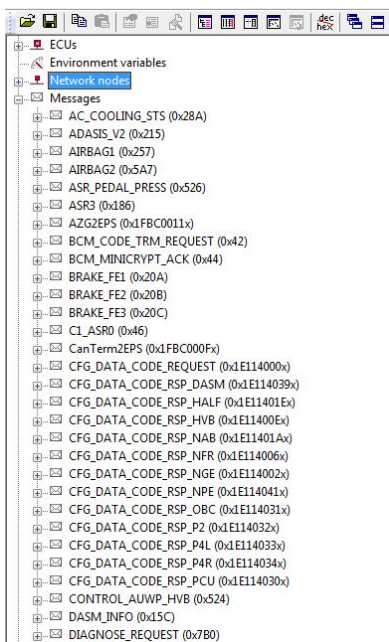
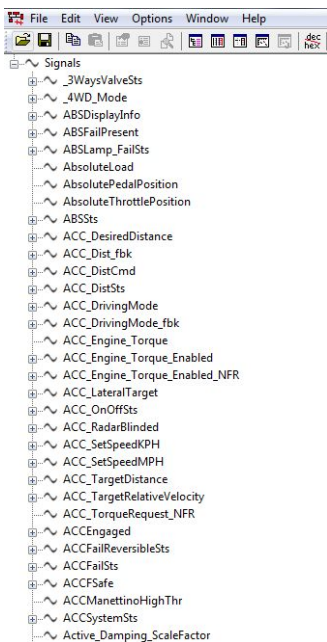
Networks
CAN
ECUs
Environment variables
Network nodes
BCM
DASM
DIAG_TESTER
HALF
HVB
NAB
NAS
NCM
NCM_SX
NCR
NFR
NQE
NPE
NQS
NTM
NVO
NVL
OBC
P2
P4L
P4R
PCU
Messages
Signals

Name	Address	Comment	TrO...	NwmNodeType	NmNodeType	ILUsed	NmNode	Nms...	NodeLay
BCM	0x40	Nodo Body Computer	0x40	kMaster	kMaster	Yes	Yes	0x40	Osek_TP.
DASM	0x2A	Driver Assistant System ...	0x2A	none*	none*	Yes	Yes	0x2A	Osek_TP.
DIAG_TESTER	0x0	Diagnostic Tester	0x0*	none*	none*	Yes*	Yes*	0x0*	Osek_TP.
HALF	0x29	Haptic Lane Feedback	0x29	none*	none*	Yes	Yes	0x29	Osek_TP.
HVB	0xA9	High Voltage Battery Hig...	0xA9	none*	none*	Yes	Yes	0xA9	Osek_TP.
NAB	0x1	Nodo AIRbag	0x1	none*	none*	Yes	Yes	0x1	Osek_TP.
NAS	0x0	Nodo Angolo Sterzo	0x0	none*	none*	Yes	Yes	0x0	Osek_TP.
NCM	0x11	Nodo Controllo Motore ...	0x11	none*	none*	Yes	Yes	0x11	Osek_TP.
NCM_SX	0x14	Nodo Controllo Motore ...	0x14	none*	none*	Yes	Yes	0x14	Osek_TP.
NCR	0x18	Nodo Cambio Robotizza...	0x18	none*	none*	Yes	Yes	0x18	Osek_TP.
NFR	0x28	Nodo Freno	0x28	none*	none*	Yes	Yes	0x28	Osek_TP.
NQE	0xE9	Nodo Guida Elettrica	0xE9	none*	none*	Yes	Yes	0xE9	Osek_TP.
NPE	0x90	Nodo Passive Entry (Rad...	0x90	none*	none*	Yes	Yes	0x90	Osek_TP.
NQS	0x85	Nodo Quadro Strumenti	0x85	none*	none*	Yes	Yes	0x85	Osek_TP.
NTM	0x83	Nodo Telemetria	0x83	none*	none*	Yes	Yes	0x83	Osek_TP.
NVO	0x8F	Nodo Volante	0x8F	none*	none*	Yes	Yes	0x8F	Osek_TP.
NVL	0x0	Nodo Sensore Imbardata	0x0	none*	none*	Yes	Yes	0x0	Osek_TP.
OBC	0xFB	On Board Charger	0xFB	none*	none*	Yes	Yes	0xFB	Osek_TP.
P2	0xAB	M E posteriore	0xAB	none*	none*	Yes	Yes	0xAB	Osek_TP.
P4L	0xAC	M E anteriore sx	0xAC	none*	none*	Yes	Yes	0xAC	Osek_TP.
P4R	0xAD	M E anteriore dx	0xAD	none*	none*	Yes	Yes	0xAD	Osek_TP.
PCU	0xAA	DCDC converter HV/LV	0xAA	none*	none*	Yes	Yes	0xAA	Osek_TP.



CAN - Network Specification DBC

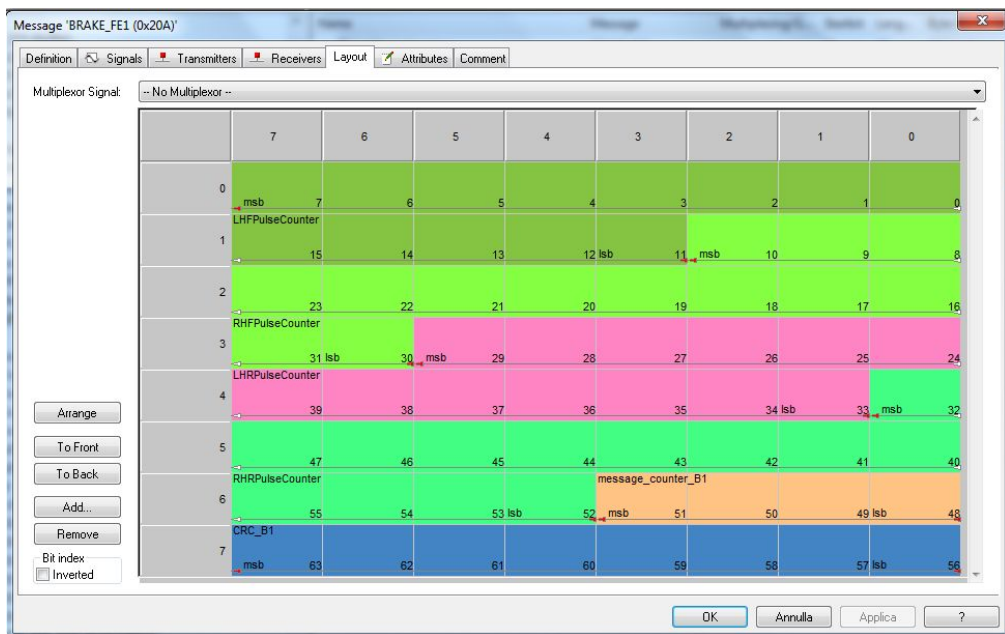
List of the messages and signals present on network





CAN - Network Specification DBC

For each message is also specified the layout of the data field.





CAN - Common CAN type

The main CAN bus that are used on the vehicle are classified on the basis of the baud rate.

The most used network are:

- C-CAN: the baud rate is 500 kb/s;
- B-CAN: the baud rate is 50 kb/s;
- BH-CAN: the baud rate is 125 kb/s;
- FD-CAN: the baud rate is variable up to 15Mb/s.

Usually on the C-CAN there are for sure all the ECUs that are emission relevant in accord to EOBD normative.

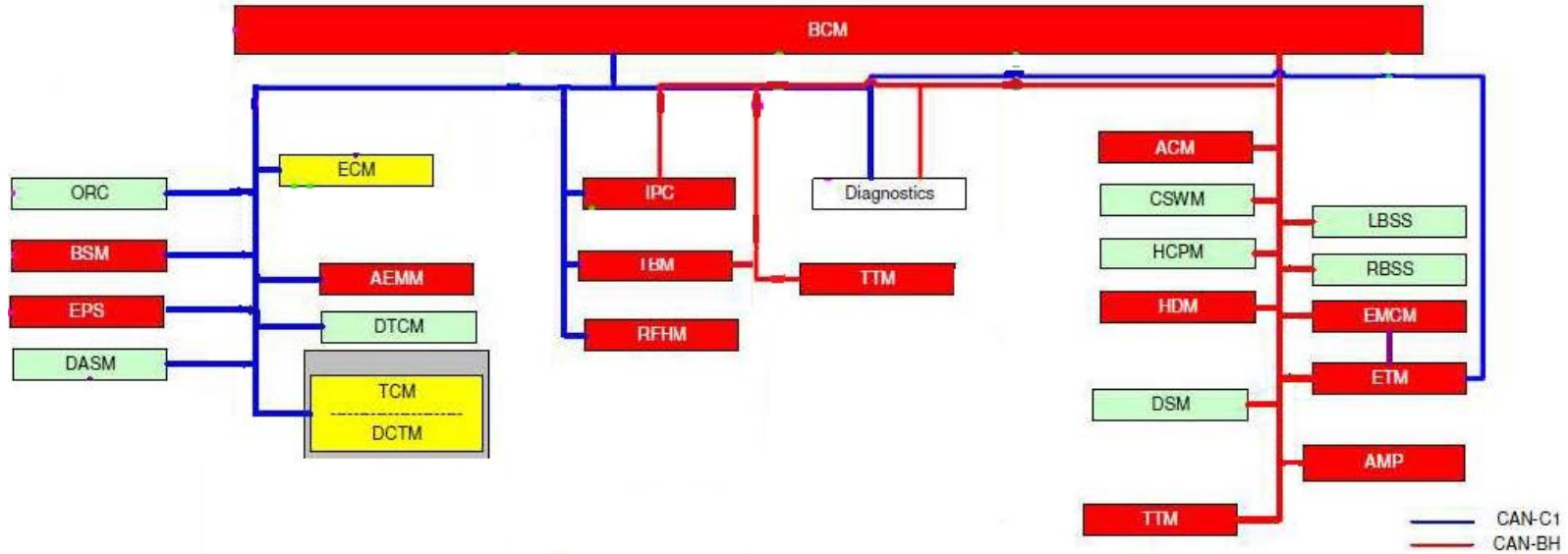
Usually on the BH-CAN there are all the ECUs that manage multimedial information

Bit rate	bit time	bus length
1 Mb/s	1 μ s	25m
800 kb/s	1.25 μ s	50m
500 kb/s	2 μ s	100m
250 kb/s	4 μ s	250m
125 kb/s	8 μ s	500m
62.5 kb/s	16 μ s	1000m
20 kb/s	50 μ s	2500m
10 kb/s	100 μ s	5000m



CAN - Main vehicle ECU

In the figure below there are an example of CAN networks for a vehicle with the main ECUs.





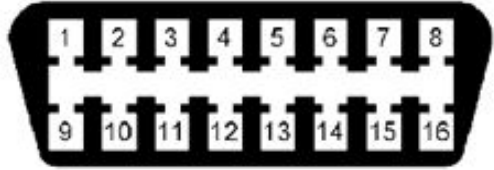
CAN - Access to CAN bus

It's possible to access on the CAN bus by the diagnostic socket also known as EOBD connector.

This connector usually is fitted on the driver side of the vehicle between driver door and steering wheel.

The features of this connector are:

- it has 16 pin;
- the pin 6 and 14 are reserved for EOBD CAN;
- the pin 4 and 5 are reserved for ground;
- the pin 16 is reserved for Power supply (+12V);
- the other pin are configurable from vendor.



PIN	DESCRIPTION	PIN	DESCRIPTION
1	Vendor Option	9	Vendor Option
2	J1850 Bus +	10	J1850 Bus -
3	Vendor Option	11	Vendor Option
4	Chassis Ground	12	Vendor Option
5	Signal Ground	13	Vendor Option
6	CAN (J-2234) High	14	CAN (J-2234) Low
7	ISO 9141-2 K-Line	15	ISO 9141-2 L-Line
8	Vendor Option	16	Battery Power

OBD-II Connector and Pinout



LIN – Local Interconnect Network



LIN -

Local Interconnect Network

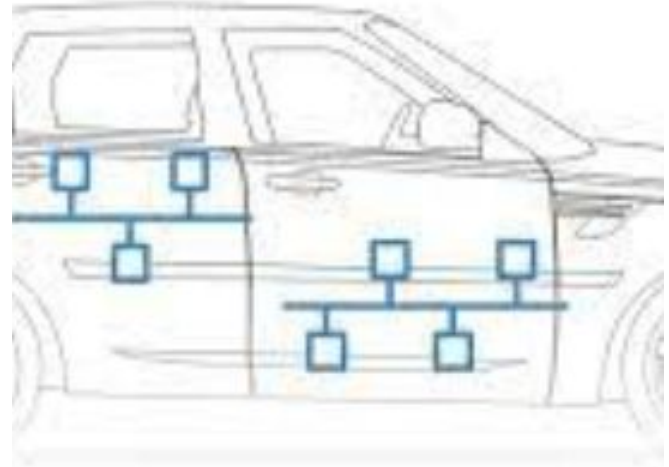
LIN (**Local Interconnect Network**) is a serial network protocol used for communication between components in vehicles.

It is used typically for nodes that do not need excessive bandwidth, performance, or extreme safety requirements.

LIN is a broadcast serial network, comprising one **master** and typically up to 16 **slaves**.

All messages are initiated by the master with at most one slave replying to a given message identifier.

LIN Network is mainly used for smart actuators and sensors, that need to transmit/receive less information with less frequency.





LIN -

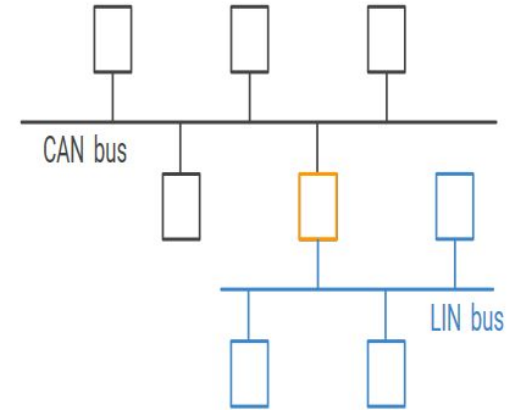
Differences and integration with CAN Network

LIN provides low cost efficient communication in applications where the bandwidth and versatility of CAN are not required.

Typically, it is used for sub-systems that are not critical to vehicle performance or safety.

- Lin Network is easier to realize and cheaper than CAN Network because it has only one wire vs. double wire of CAN Network. It is also easier to configure, and to implement additional extensions. Since there is one master of the network, no collision strategies are needed.
- Can network is more robust to disturbance and reaches higher speed than LIN Network

The LIN master has also a CAN connection, and performs gateway functions.





LIN -

Protocol main features

- Single master, up to 16 slaves
- Slave Node Position Detection (SNPD) allows node address assignment after power-up
- Single wire communications up to 19.2 kbit/s for 40 meter bus length.
- Guaranteed latency times.
- Variable length of data frame (2, 4 and 8 byte).
- Data checksum and error detection.
- Detection of defective nodes.
- Operating voltage of 12 V
- Standard compliant: ISO 9141



LIN -

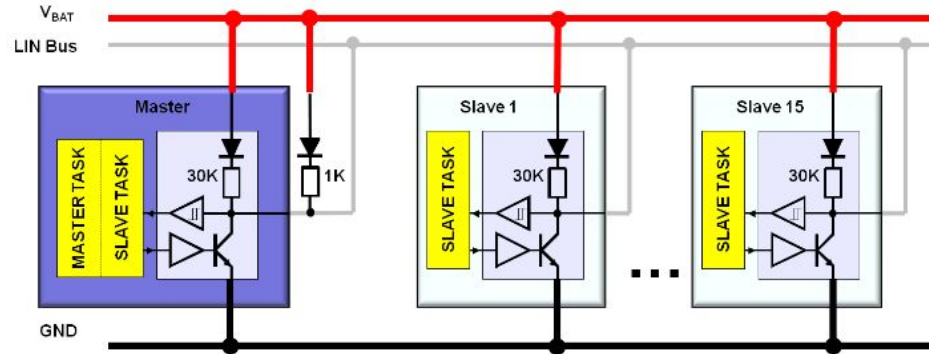
Hardware features and physical layer

LIN Bus is connected to the 12V power supply with a pull up Resistance 1 k Ω in the master node, and typically 30 k Ω in a slave node. Series diodes are connected to avoid overvoltage.

The connection to the ground Can also be on vehicle chassis.

The bus operates with two complementary logic levels: The **dominant** value with a voltage close to ground represents a logical 0.

The **recessive** value with an electrical voltage close to the battery voltage, and represent a logical 1





LIN -

Master and Slaves

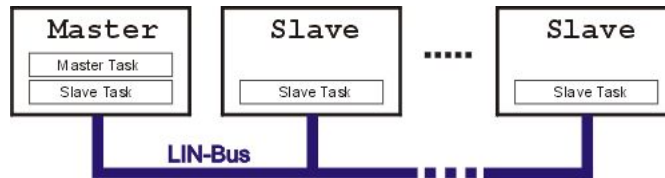
The **Master** has the complete control of both the bus, on which the data are entered, and the communication protocol.

Determines when to initiate message transfer to all slave nodes.

It determines when to start message transfer on the bus, send the Synch Break (predefined sequence of bytes preceding a frame) and receives wake up Break bits from Slaves when they request to communicate.

Performs the monitoring of the data and check byte, verifying their consistency and correctness respectively.

The **Slave** is one of the nodes present on the bus that can receive or transmit data when the Master decide it. It synchronizes on the Synch Byte issued by the Master, analyze the identifier-field bits and based on the content of the identifier field, determines what to do.



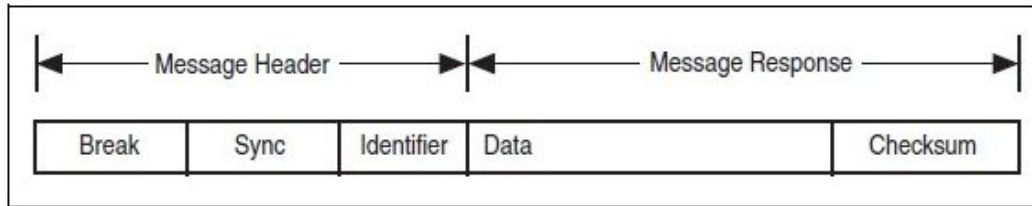


LIN -

Message frames

Data is transferred across the bus in fixed form messages of selectable lengths. The master transmits a message header, that consists of fields, that synchronize and identify it. The slaves respond with a data frame that consists of data and checksum.

- **Message Header** □ it consists of a break, used to identify the start of frame, a sync field, used by the slave node for clock synchronization, and the identifier that denotes a specific message address, but not the destination.
- **Message Response** □ upon the reception of the ID, the polled slave begins the message response which consists of 1 to 8 bytes of data, and 8-bit checksum.



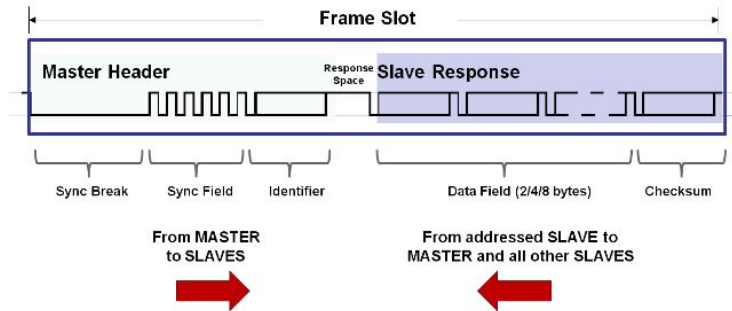


LIN -

Message frames

Upon receiving the header, each slave checks the ID to determine what it needs to publish or subscribe.

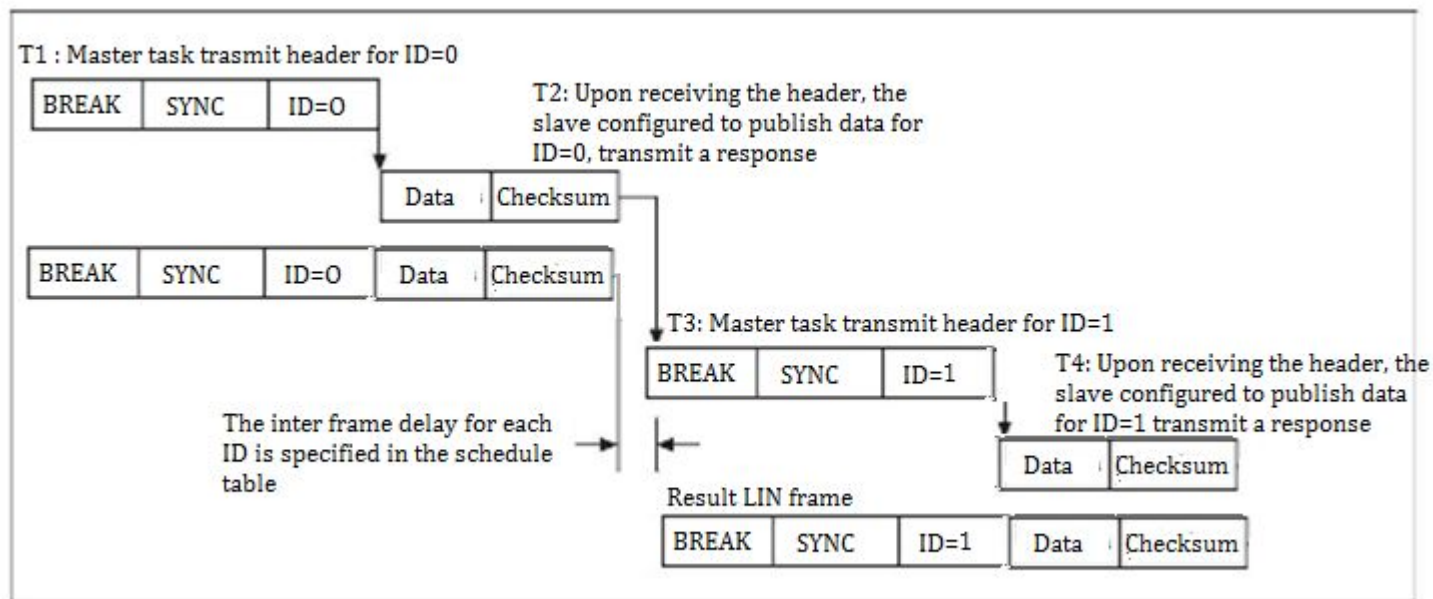
If the slave task needs to publish a response, it transmits one to eight data bytes to the bus followed by a checksum byte.



The master controls the sequencing of message frames, which is defined in a schedule that can be changed as needed. The Master uses one or more predefined **scheduling tables** to start the sending and receiving on the LIN bus. These scheduling tables contain relative timing, where the message sending is initiated.



LIN - Message frames





LIN -

Scheduling Tables

A key property of the LIN protocol is the use of **schedule tables**.

Schedule tables make it possible to assure that the bus will never be overloaded and to guarantee the periodicity of signals.

The schedule table in the LDF defines the sequence and time grid in which messages are sent. Once the table has been worked through, the Master begins with the first message again.

It's possible that several schedule tables co-exist in a LIN. When a schedule table has been performed, any one of the rest of schedule tables may be performed. It depends on system policies.

The idea to support different type of schedule is that there is always a set of schedule that runs continuously, in normal operation, but if there is a specific situation or request, another schedule can be started.



LIN -

Network Specifications: LDF

The configuration of the LIN network is described in the **LIN Description File LDF**.

In the LDF all LIN messages with correspondent signals are described, and it contains also features of the network, such as the number of nodes present, and in which schedule the messages appears, with also their periodicity.

For example, it defines the baud rate, the ordering and time delays for the master task's transmission of headers, and the behavior of each slave task in response.

Each LIN present, has a dedicated LDF File.

Frames/Commands

Nodes/Frames

Ferrari_NBC-MCL-E_Latch_LH_WLML_SignalList_LIN2_F173_rev_2

Unconditional Frames

NBCToEL_LH_Commands (5)

BlatchLH_Status (7)

NBC_MEM_POS_EXMI_LIN (45)

Fdbk_Pos_Exmi_LH (17)

NBC_CNTRL_EXMI_LIN (47)

Er_LH_Exmi_Sts (15)

Er_Warn_LH_Exmi_LIN (48)

NBCToWLMLCommand (24)

WLMLToNBCFeedback (25)

Diagnostic Frames

MasterReq (60)

SlaveResp (61)

Event-Triggered Frames

Sporadic Frames

Table / Slot

Type

Delay [ms]

Max. Frame Time [ms]

Min. Frame Time [ms]

Act. Min/Max Period [ms]

EL_Complete

NBCToEL_LH_Commands (5)

BlatchLH_Status (7)

EL_KeyOFF

NBCToEL_LH_Commands (5)

BlatchLH_Status (7)

WLML_KeyOFF

NBCToEL_LH_Commands (5)

BlatchLH_Status (7)

NBCToWLMLCommand (24)

WLMLToNBCFeedback (25)

WLML

NBCToWLMLCommand (24)

Unconditional

Unconditional

Unconditional

Unconditional

Unconditional

Unconditional

Unconditional

Unconditional

Unconditional

Unconditional

Unconditional

Unconditional

10.00

10.00

10.00

10.00

10.00

10.00

10.00

10.00

10.00

10.00

4.66

6.12

4.66

6.12

4.66

6.12

9.05

5.39

9.05

3.33

4.37

3.33

4.37

3.33

4.37

6.46

3.85

6.46

20.00 / 20.00

20.00 / 20.00

20.00 / 20.00

20.00 / 20.00

40.00 / 40.00

40.00 / 40.00

40.00 / 40.00

40.00 / 40.00

40.00 / 40.00

20.00 / 20.00

Schedule Table Properties

Name

Table Id

Bus Load [%]

Total Cycle Time [ms]

Resolves collisions for ETP(s)

EL_Complete

0

53.9

20.0

-



LIN -

Network Specifications: LDF

Frames/Commands

Nodes/Frames

Ferrari_LIN_SignalList_F142MVS_2017-11-10

NCM

- Published Signals
- Subscribed Signals
- Published Frames
- Subscribed Frames

LISB (11)

- Published Signals
- Subscribed Signals
- Published Frames
- Subscribed Frames

DCDC_CON (106)

- Published Signals
- Subscribed Signals
- Published Frames
- Subscribed Frames

PICO (3)

- Published Signals
- Subscribed Signals
- Published Frames
- Subscribed Frames

SA (1)

- Published Signals
- Subscribed Signals
- Published Frames
- Subscribed Frames

DIAG_TESTER (64)

- Published Signals
- Subscribed Signals
- Published Frames
- Subscribed Frames

Schedule Tables

Table / Slot

Type

Delay [ms]

Max. Frame Time [ms]

Min. Frame Time [ms]

Act. Min/Max Period [ms]

Schedule Table Properties

Nodes of <Ferrari_LIN_SignalList_F142MVS_2017-11-10_v1>

Node	/	Timebase [ms]	Jitter [ms]	Configured NAD	Initial NAD	Protocol	Supplier ID	Function ID	Variant ID
DCDC_CON		-	-	106	-	2.0	0	0	0
DIAG_TESTER		-	-	64	-	1.3	-	-	-
LISB		-	-	11	11	2.1	119	1	1
NCM		10	0.0	-	-	-	-	-	-
PICO		-	-	3	-	2.0	0	0	0
SA		-	-	1	-	1.3	-	-	-

Common

Protocol Version 2.2

Language Version 2.2

LIN Bus Properties

Name Ferrari_LIN_SignalList

Baudrate 19.200 kbps

Channel Postfix Sig_Type

Comment /



LIN -

Scheduling Tables

Frames/Commands

Nodes/Frames

- Ferrari_NBC-MCL-E_Latch_LH_WLML_Signa
 - Unconditional Frames
 - NBCToEL_LH_Commands (5)
 - ElatchLH_Status (7)
 - NBC_MEM_POS_EXMI_LIN (45)
 - Fdbk_Pos_Exmi_LH (17)
 - NBC_CNTRL_EXMI_LIN (47)
 - Er_LH_Exmi_Sts (15)
 - Er_Wam_LH_Exmi_LIN (48)
 - NBCToWLMCommand (24)
 - WLMLToNBCFeedback (25)
 - Diagnostic Frames
 - MasterReq (60)
 - SlaveResp (61)
 - Event-Triggered Frames
 - Sporadic Frames
 - Predefined Commands

Schedule Tables

Table / Slot	Type	Delay [ms]	Max. Frame Time [ms]	Min. Frame Time [ms]	Act. Min/Max Period [ms]
EL_Complete - NBCToEL_LH_Commands (5) - ElatchLH_Status (7)	Unconditional	10.00	4.66	3.33	20.00 / 20.00
EL_KeyOFF - NBCToEL_LH_Commands (5) - ElatchLH_Status (7)	Unconditional	10.00	6.12	4.37	20.00 / 20.00
WLM_El_KeyOFF - NBCToEL_LH_Commands (5) - ElatchLH_Status (7) - NBCToWLMCommand (24) - WLMLToNBCFeedback (25)	Unconditional	10.00	4.66	3.33	20.00 / 20.00
WLM - NBCToWLMCommand (24) - WLMLToNBCFeedback (25)	Unconditional	10.00	9.05	6.46	40.00 / 40.00
	Unconditional	10.00	5.39	3.85	40.00 / 40.00
	Unconditional	10.00	9.05	6.46	20.00 / 20.00
	Unconditional	10.00	5.39	3.85	20.00 / 20.00

General Slot Properties

Delay 10.00

Assigned Frame / Command

Frame	NBCToEL_LH_Comma...
Min Frame Time	3.33
Max Frame Time	4.66
Min Max Period	20.00 / 20.00

Signals of Unconditional Frame <NBCToEL_LH_Commands>

Signal	Startbit	/	Publisher	Subscribers	Length [Bit]	Init Value	Unit	Encoding
NBC_LH_Lock	0		NBC	ELL	1	0		logical
NBC_LH_Unlock	1		NBC	ELL	1	0		logical
NBC_LH_CrashSignal	2		NBC	ELL	1	0		logical
NBC_LH_CrashSignal_ValidData	3		NBC	ELL	1	0		logical
NBC_LH_LED_Activation	4		NBC	ELL	2	0		logical
NBC_LH_LED_Intensity	6		NBC	ELL	1	0		logical
NBC_LH_VehideSpeedFailSts	7		NBC	ELL	1	0		logical
NBC_LH_VehideSpeedSts	8		NBC	ELL	2	0		logical
NBC_LH_PowerModeSts	10		NBC	ELL	2	0		logical
EL_NPE_Connection_Check_Req	12		NBC	ELL	1	0		logical

Signal Encodings

Name	NBC_LH_Lock_Enc...
Physical	
Logical	0: Not Active, 1: Ac...

Signal Properties

Name	NBC_LH_Lock
Length	1
Init Value	0
Publisher	NBC
Subscribers	ELL
Startbit	0
Frame	NBCToEL_LH_Com...
Type	Scalar
Unit	



LIN -

Frames types

- **Unconditional frame** □ This is the normal type of LIN communication. The Master sends a frame header in a scheduled frame slot and the designated Slave node fills the frame with data. Unconditional frames carry signals and their frame identifiers are in the range 0 (0x00) to 59 (0x3B).
- **Event-triggered frame** □ The purpose of this is to increase the responsiveness of the LIN cluster without assigning too much of the bus bandwidth to the polling of multiple slave nodes. The first data byte of the carried unconditional frame shall be equal to a protected identifier assigned to an event-triggered frame. A slave shall reply with an associated unconditional frame only if its data value has changed. If none of the slave tasks responds to the header the rest of the frame slot is silent and the header is ignored. If more than one slave task responds to the header in the same frame slot a collision will occur, and the master has to resolve the collision by requesting all associated unconditional frames before requesting the event-triggered frame again.



LIN -

Frames types

Unconditional Frame example

The screenshot displays a LIN configuration interface. On the left, a tree view shows the hierarchy of frames, with 'Unconditional Frames' expanded to show 'VehData_Frm_AR (10)' selected. Below it are various diagnostic and event-triggered frames. The main area on the right shows the 'Signals of Unconditional Frame <VehData_Frm_AR>' table.

Signal	Startbit	/	Publisher	Subscribers	Length [Bit]	Init Value	Unit	Encoding
Isw_Stat	0		NCM	LISB	3	7		logical
EngRun_Stat	3		NCM	LISB	3	7		logical
StopStartSts	6		NCM	LISB	2	0		logical
LISBRelayDiagEnable	8		NCM	LISB	1	0		logical



LIN -

Frames types

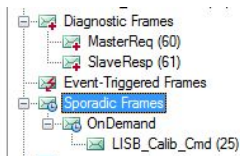
- **Sporadic frame** □ Sent by Master only when it is known that a Slave has updated information to publish. The header of a sporadic frame shall only be sent in its associated frame slot when the master knows that a signal carried in the frame has been updated. The publisher of the sporadic frame shall always provide the response to the header.
- **Diagnostic frame** □ These always carry diagnostic or configuration data and they always contain eight data bytes. The identifier is either 0x3C, called master request frame, or 0x3D, called slave response frame. Before generating the header of a diagnostic frame, the master asks its diagnostic module if it shall be sent or if the bus shall be silent. The slave publish and subscribe to the response according to their diagnostic module.
- **User-defined frame** □ These can carry any kind of information. Their identifier is 62 (0x3E). The header of a user-defined frame is always transmitted when a frame slot allocated to the frame is processed



LIN -

Frames types

Sporadic Frame example



Sporadic Frames of <Ferrari_LIN_SignalList_F142MVS_2017-11-10_v1>				
Frame	/ ID	Length [Byte]	Publisher	Subscribers
OnDemand			NCM	LISB

Diagnostic Frame example



Diagnostic Frames of <Ferrari_LIN_SignalList_F142MVS_2017-11-10_v1>				
Frame	/ ID	Length [Byte]	Publisher	Subscribers
MasterReq		60 [60]	8 NCM (Master node)	Each slave node
SlaveResp		61 [125]	8 Each slave node	NCM (Master node)



LIN -

Sleep and WakeUp

LIN network has a mechanism that allows devices to enter the sleep mode and so conserve power. All slaves may be forced into sleep mode by the master sending a diagnostic master request frame with the first data byte equal to zero.

This special frame is called the **go-to-sleep command**.

Slaves also automatically enter sleep mode if the LIN is inactive for more than four seconds.

LIN also has a mechanism for waking devices on the bus.

Wakeup is one task that may be initiated by any node on the bus (a slave as well as the master).

Each slave should detect the wakeup request and be ready to process headers within 100 ms.

The master should also detect the wakeup request and start sending headers when the slave nodes are ready (within 100 ms to 150 ms after receiving the wakeup request).

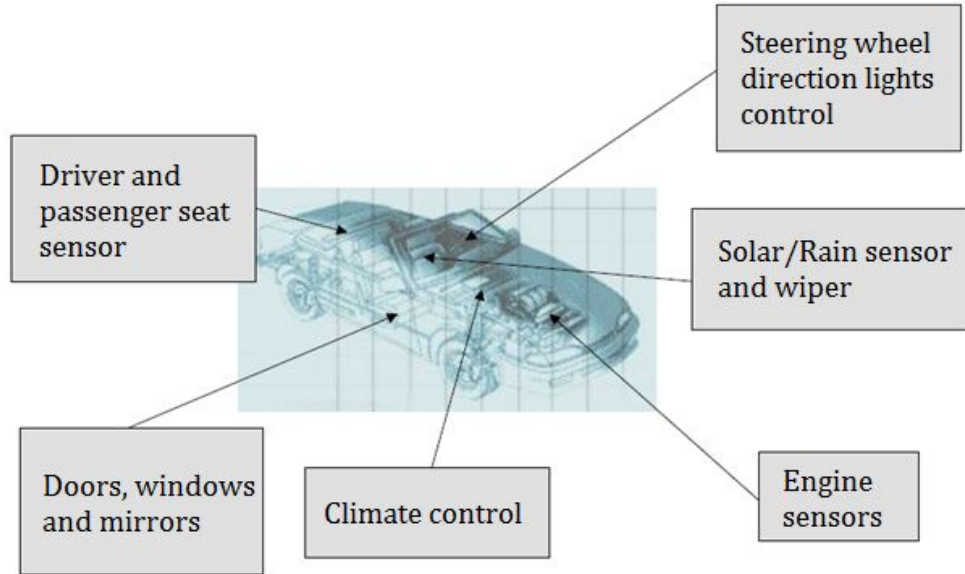
If the master does not issue headers within 150 ms after receiving the first wakeup request, then the slave shall try again up to 4 times, before to detect an error.



LIN -

Examples and applications

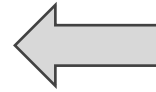
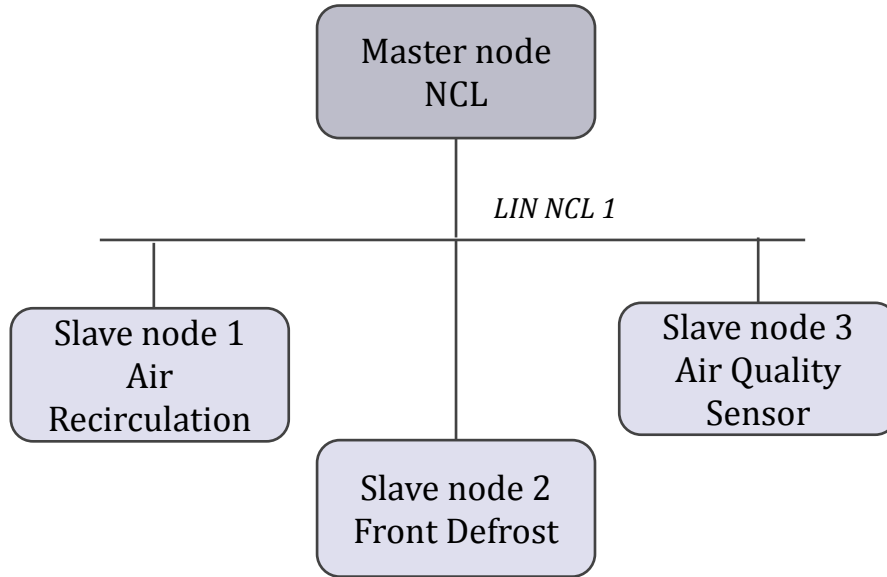
- Power door locks
- Power windows
- Power seats
- Power mirrors
- Windshield wipers
- Seat heaters
- Heating and air-conditioning controls
- Interior lights
- Climate controls
- Steering-wheel controls
- Capote
- Trunk





LIN -

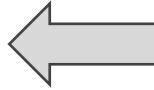
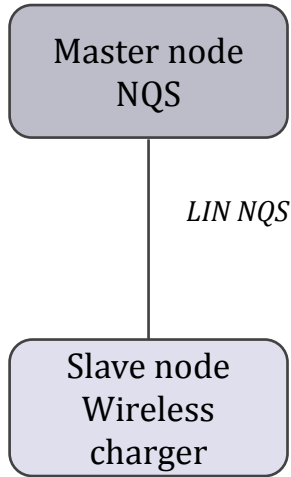
Examples and applications





LIN -

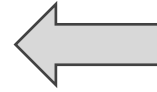
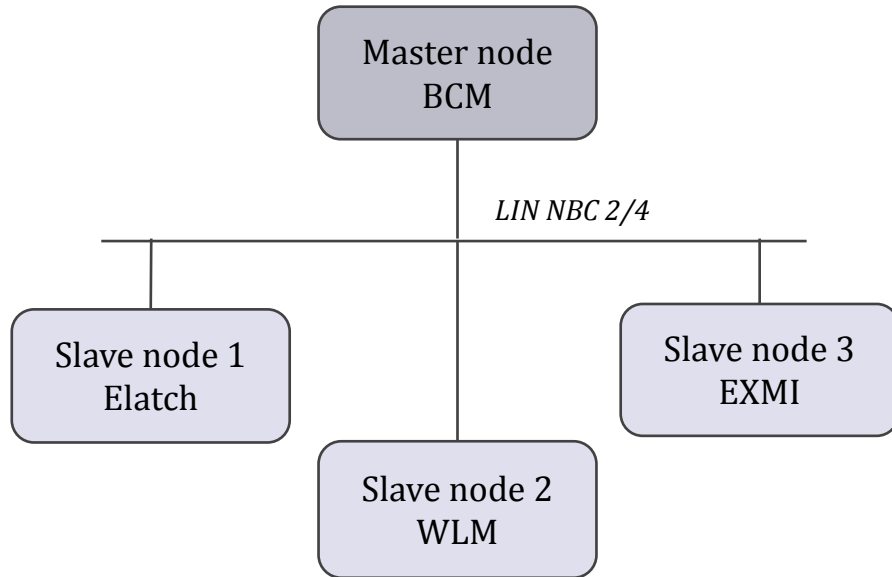
Examples and applications





LIN -

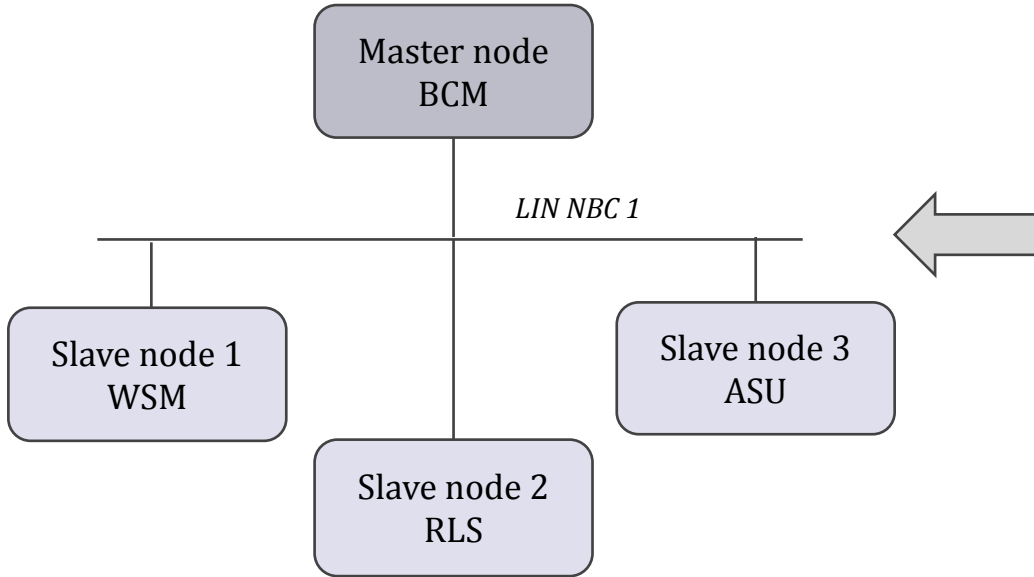
Examples and applications





LIN -

Examples and applications





Ethernet for automotive

CONTACTS

Bruno D'Angelo

bruno.dangelo@kineton.it

Pasqualina Verolino

pasqualina.verolino@kineton.it

Via G. Porzio,
Centro Direzionale Isola B3
80143 Napoli Via G. Porzio

www.kineton.it

